```
    end
  end

  # Do the same after finding an existing record
  alias_method :after_find, :after_save
end
```

This example uses trivial encryption—you might want to beef it up before using this class for real.

We can now arrange for the `Encrypter` class to be invoked from inside our orders model:

```
require "encrypter"
class Order < ApplicationRecord
  encrypter = Encrypter.new([:name, :email])
  before_save encrypter
  after_save  encrypter
  after_find  encrypter
protected
  def after_find
  end
end
```

We create a new `Encrypter` object and hook it up to the events `before_save`, `after_save`, and `after_find`. This way, just before an order is saved, the method `before_save` in the encrypter will be invoked, and so on.

So why do we define an empty `after_find` method? Remember that we said that for performance reasons `after_find` and `after_initialize` are treated specially. One of the consequences of this special treatment is that Active Record won't know to call an `after_find` handler unless it sees an actual `after_find` method in the model class. We have to define an empty placeholder to get `after_find` processing to take place.

This is all very well, but every model class that wants to use our encryption handler would need to include some eight lines of code, just as we did with our `Order` class. We can do better than that. We'll define a helper method that does all the work and make that helper available to all Active Record models. To do that,